

TALENTPREDICT - ANALYSE COMPLÈTE DU PROJET

Date d'analyse: 5 Mars 2026

Version: 1.0

Chemin projet: C:\Users\rahma\Desktop\TalentPredict

SECTION 1: PROJECT OVERVIEW

Objectif du Projet

TalentPredict est une plateforme d'évaluation des compétences et de prédiction de formations basée sur l'intelligence artificielle (OpenAI GPT-4). Elle permet aux entreprises de gérer les talents, d'évaluer les personnalités (tests PCM, MBTI, DISC), de suivre les compétences techniques et soft skills, et de proposer des formations personnalisées.

Problème Métier Résolu

- **Identification des lacunes de compétences** : Analyse des soft skills et tech skills des employés
- **Prédiction IA** : Recommandations de formations basées sur les tests de personnalité et compétences
- **Suivi des formations** : Progression, statuts et validation par l'administration
- **Gestion des talents** : Dashboard RH pour piloter les évaluations et formations
- **Automatisation** : Création de tickets Jira pour le suivi des formations

Utilisateurs Cibles et Rôles

Rôle	Type	Fonctionnalités
USER (Employé)	Standard	☒ Passer des tests de personnalité prédictions IA ☒ Dashboard personnel
ADMIN (RH/Manager)	Administrateur	☒ Gérer ses compétences (skills) ☒ Consulter ses formations proposées ☒ Voir ses formations ☒ Toutes les fonctionnalités USER ☒ Créer/modifier/supprimer des comptes ☒ Valider les compétences des employés ☒ Voir tous les tests et prédictions ☒ Dashboard RH (vue globale) ☒ Gérer les tickets Jira

Résumé des Fonctionnalités Principales

1. **Authentification JWT** avec rôles USER/ADMIN
2. **Tests de personnalité** (PCM, MBTI, DISC) avec analyse OpenAI
3. **Gestion des compétences** : Soft skills et Tech skills (niveaux 1-5)
4. **Prédictions IA** : Recommandations de formations par OpenAI
5. **Suivi des formations** : Progression et statuts (Proposée → En cours → Terminée)
6. **Intégration Jira** : Création automatique de tickets de formation
7. **Dashboard** : Vue employé et vue RH
8. **Profils utilisateurs** : CV, LinkedIn, GitHub, expérience
9. **Reset password** : Système de récupération par email

SECTION 2: TECH STACK

Backend (Spring Boot)

Technologie	Version	Détails
Framework	Spring Boot	3.4.2
Java	JDK	21
ORM	Spring Data JPA	3.4.2 (Hibernate)
Database	PostgreSQL	runtime (driver included)
Security	Spring Security + JWT	io.jsonwebtoken:jjwt 0.12.6
Validation	Spring Validation	Jakarta Validation API
Email	Spring Mail	For password reset
AI Library	OpenAI GPT-4	theokanning.openai-gpt3-java:service 0.18.2
Utilities	Lombok	1.18.38
Testing	JUnit + Spring Test	spring-boot-starter-test
Build Tool	Maven	3.13.0 (compiler plugin)

Frontend (Angular)

Technologie	Version	Détails
Framework	Angular	21.1.0
TypeScript	TypeScript	5.9.2
CSS Framework	Tailwind CSS	4.1.12
HTTP Client	RxJS	7.8.0
Testing Framework	Vitest	4.0.8
Linting	ESLint + Angular ESLint	9.39.2 / 21.2.0
SSR	Angular SSR	21.1.2
Server	Express	5.1.0 (for SSR)

Base de Données

- **Type:** PostgreSQL (Relational Database)
- **Version:** Non spécifiée (runtime postgresql driver)
- **ORM:** Hibernate via Spring Data JPA

SECTION 3: ALL ENTITIES & TABLES

1. Account (accounts)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
username	username	String	UNIQUE, LENGTH 50
password	password	String	NOT NULL
email	email	String	UNIQUE, NOT NULL, LENGTH 255
isActive	is_active	Boolean	NOT NULL, DEFAULT true
role	role	Account.Role (ENUM)	NOT NULL, LENGTH 20, DEFAULT USER
firstName	first_name	String	NOT NULL, LENGTH 100
lastName	last_name	String	NOT NULL, LENGTH 100
department	department	String	LENGTH 100
position	position	String	LENGTH 100
hireDate	hire_date	LocalDate	
profilePictureUrl	profile_picture_url	String	LENGTH 500
createdAt	created_at	Instant	NOT NULL, AUTO (CreationTimestamp)
updatedAt	updated_at	Instant	AUTO (UpdateTimestamp)
profile	-	Profile	OneToOne (mappedBy)
tests	-	List<PersonalityTest>	OneToMany (mappedBy)
skills	-	List<Skill>	OneToMany (mappedBy)
predictions	-	List<Prediction>	OneToMany (mappedBy)
formations	-	List<Formation>	OneToMany (mappedBy)
recommendations	-	List<Recommendation>	OneToMany (mappedBy)
competences	-	List<CompetenceAccount>	OneToMany (mappedBy)

Enum Account.Role: USER, ADMIN

2. Profile (profiles)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
titreProfessionnel	titre_professionnel	String	LENGTH 200
description	description	String	TEXT
urlPhoto	url_photo	String	LENGTH 500
experienceAns	experience_ans	Integer	
niveauEtudes	niveau_etudes	String	LENGTH 100
lienLinkedin	lien_linkedin	String	LENGTH 500
githubUrl	github_url	String	LENGTH 500
cvUrl	cv_url	String	LENGTH 500
accountId	account_id	UUID	FK → Account, UNIQUE, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)
pcmResults	-	List<PCMResult>	OneToMany (mappedBy)
createdAt	created_at	Instant	NOT NULL, AUTO
updatedAt	updated_at	Instant	AUTO

3. Skill (skills)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
nom	nom	String	NOT NULL, LENGTH 200
type	type	Skill.TypeSkill (ENUM)	NOT NULL, LENGTH 20
niveau	niveau	Integer	NOT NULL (1-5)
description	description	String	TEXT
dateEvaluation	date_evaluation	LocalDateTime	DEFAULT now()
validee	validee	Boolean	DEFAULT false
accountId	account_id	UUID	FK → Account, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)

Enum Skill.TypeSkill: SOFT, TECH

4. Formation (formations)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
titre	titre	String	

			NOT NULL, LENGTH 300
description	description	String	TEXT
duree	duree	Integer	(hours)
fournisseur	fournisseur	String	LENGTH 200
url	url	String	LENGTH 500
dateProposition	date_proposition	LocalDateTime	DEFAULT now()
dateDebut	date_debut	LocalDateTime	
dateFin	date_fin	LocalDateTime	
progression	progression	Integer	DEFAULT 0 (0-100)
type	type	Formation.TypeFormation (ENUM)	NOT NULL, LENGTH 30
statut	statut	Formation.StatutFormation (ENUM)	NOT NULL, LENGTH 30, DEFAULT PROPOSEE
predictionId	prediction_id	UUID	FK → Prediction
accountId	account_id	UUID	FK → Account, NOT NULL
prediction	-	Prediction	ManyToOne (FetchType.LAZY)
account	-	Account	ManyToOne (FetchType.LAZY)
tickets	-	List<Ticket>	OneToMany (mappedBy)

Enum Formation.TypeFormation: SOFT_SKILL, TECH_SKILL, CERTIFICATION, WORKSHOP

Enum Formation.StatutFormation: PROPOSEE, ACCEPTEE, EN_COURS, TERMINEE, ANNULEE

5. Inscription (inscriptions)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
dateInscription	date_inscription	LocalDate	DEFAULT today()
statut	statut	Inscription.StatutInscription (ENUM)	NOT NULL, LENGTH 30, DEFAULT EN_COURS
progression	progression	Integer	DEFAULT 0
dateFin	date_fin	LocalDate	
noteFinale	note_finale	Double	
accountId	account_id	UUID	FK → Account, NOT NULL
formationId	formation_id	UUID	FK → Formation, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)
formation	-	Formation	ManyToOne (FetchType.LAZY)
createdAt	created_at	Instant	NOT NULL, AUTO
updatedAt	updated_at	Instant	AUTO

Enum Inscription.StatutInscription: EN_COURS, TERMINEE_ANNULEE, SUSPENDUE

6. Ticket (tickets)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
jiraKey	jira_key	String	UNIQUE, LENGTH 50 (ex: TRN-123)
titre	titre	String	NOT NULL, LENGTH 300
description	description	String	TEXT
statut	statut	Ticket.StatutTicket (ENUM)	NOT NULL, LENGTH 30, DEFAULT OUVERT
priorite	priorite	Ticket.PrioriteTicket (ENUM)	LENGTH 20, DEFAULT MOYENNE
assignee	assignee	String	LENGTH 200
urlJira	url_jira	String	LENGTH 500
formationId	formation_id	UUID	FK → Formation, NOT NULL
formation	-	Formation	ManyToOne (FetchType.LAZY)
createdAt	created_at	Instant	NOT NULL, AUTO
updatedAt	updated_at	Instant	AUTO

Enum Ticket.StatutTicket: OUVERT, EN_COURS, EN_ATTENTE, RESOLU, FERME

Enum Ticket.PrioriteTicket: BASSE, MOYENNE, HAUTE, CRITIQUE

7. PersonalityTest (tests_personnalite)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
dateTest	date_test	LocalDateTime	NOT NULL, DEFAULT now()
typeTest	type_test	String	LENGTH 50 (PCM, MBTI, DISC, etc.)
reponses	-	Map<String, String>	@ElementCollection → table test_reponses
resultats	resultats	String	TEXT
analyseLlm	analyse_llm	String	TEXT (OpenAI analysis)
score	score	Integer	
accountId	account_id	UUID	FK → Account, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)

@ElementCollection Table: test_reponses

- test_id (FK → tests_personnalite)
- question_key (String)
- response_value (TEXT)

8. PCMResult (pcm_results)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
typePCM	type_pcm	TypePCM (ENUM)	LENGTH 30
scoreTravail	score_travail	Integer	
scoreSecondaire	score_secondaire	Integer	
scoreReactif	score_reactif	Integer	
scoreRebelle	score_rebelle	Integer	
dateEvaluation	date_evaluation	LocalDateTime	DEFAULT now()
profileId	profile_id	UUID	FK → Profile, NOT NULL
profile	-	Profile	ManyToOne (FetchType.LAZY)
createdAt	created_at	Instant	NOT NULL, AUTO
updatedAt	updated_at	Instant	AUTO

Enum TypePCM: ANALYTIQUE, AMIABLE, DIRECTIF, EXPRESSIF

9. Competence (competences)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
nom	nom	String	NOT NULL, LENGTH 200
categorie	categorie	String	NOT NULL, LENGTH 100
description	description	String	TEXT
niveau	niveau	Competence.Niveau (ENUM)	NOT NULL, LENGTH 30, DEFAULT DEBUTANT
dateEvaluation	date_evaluation	LocalDateTime	
dateCreation	date_creation	LocalDateTime	DEFAULT now()
dateModification	date_modification	LocalDateTime	DEFAULT now()
utilisateurs	-	List<CompetenceAccount>	OneToMany (mappedBy)

Enum Competence.Niveau: DEBUTANT, INTERMEDIAIRE, AVANCE, EXPERT

10. CompetenceAccount (competence_account)

Table de jointure Many-to-Many entre Account et Competence avec attributs supplémentaires

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
score	score	Double	DEFAULT 0.0
dateEvaluation	date_evaluation	LocalDateTime	
dateCreation	date_creation	LocalDateTime	DEFAULT now()
dateModification	date_modification	LocalDateTime	DEFAULT now()
accountId	account_id	UUID	FK → Account, NOT NULL
competenceId	competence_id	UUID	FK → Competence, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)
competence	-	Competence	ManyToOne (FetchType.LAZY)

Unique Constraint: uk_competence_account on (account_id, competence_id)

11. Prediction (predictions)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
analyse	analyse_text	String	TEXT, NOT NULL (OpenAI analysis)
recommandationSoft	recommandation_soft	String	TEXT
recommandationTech	recommandation_tech	String	TEXT
scoreConfiance	score_confiance	Double	(0-1)
statut	statut	Prediction.StatutPrediction (ENUM)	NOT NULL, LENGTH 30, DEFAULT EN_ANALYSE
datePrediction	date_prediction	LocalDateTime	NOT NULL, DEFAULT now()
accountId	account_id	UUID	FK → Account, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)
formationsProposees	-	List<Formation>	OneToMany (mappedBy)

Enum Prediction.StatutPrediction: EN_ANALYSE, COMPLETEE, VALIDEE, APPLIQUEE

12. Recommendation (recommendations)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
titre	titre	String	LENGTH 300
description	description	String	TEXT
score	score	Double	DEFAULT 0.0
priorite	priorite	Integer	DEFAULT 1
dateGeneration	date_generation	LocalDateTime	DEFAULT now()
dateCreation	date_creation	LocalDateTime	DEFAULT now()
dateModification	date_modification	LocalDateTime	DEFAULT now()
accountId	account_id	UUID	FK → Account, NOT NULL
account	-	Account	ManyToOne (FetchType.LAZY)

13. RecommendationItem (recommendation_items)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
contenu	contenu	String	TEXT
texte	texte	String	LENGTH 500
priorite	priorite	Integer	DEFAULT 1
dateCreation	date_creation	LocalDateTime	DEFAULT now()
dateModification	date_modification	LocalDateTime	DEFAULT now()
recommendationId	recommendation_id	UUID	FK → Recommendation, NOT NULL
recommendation	-	Recommendation	ManyToOne (FetchType.LAZY)

14. PasswordResetToken (password_reset_tokens)

Field	Column Name	Java Type	Constraints
id	id	UUID	PK, NOT NULL, AUTO
token	token	String	UNIQUE, NOT NULL
accountId	account_id	UUID	FK → Account, NOT NULL
account	-	Account	ManyToOne (FetchType.EAGER)
expiryDate	expiry_date	LocalDateTime	NOT NULL
used	used	Boolean	NOT NULL, DEFAULT false

SECTION 4: ALL RELATIONS BETWEEN TABLES

📊 Tableau Récapitulatif des Relations

From Entity	To Entity	Relation Type	FK Column	Cascade	Fetch	orphanRemoval
Account	Profile	OneToOne	profile.account_id	ALL	LAZY	true
Account	PersonalityTest	OneToMany	tests_personnalite.account_id	ALL	LAZY	true
Account	Skill	OneToMany	skills.account_id	ALL	LAZY	true
Account	Prediction	OneToMany	predictions.account_id	ALL	LAZY	true
Account	Formation	OneToMany	formations.account_id	ALL	LAZY	true
Account	Recommendation	OneToMany	recommendations.account_id	ALL	LAZY	true
Account	CompetenceAccount	OneToMany	competence_account.account_id	ALL	LAZY	true
Account	Inscription	OneToMany	inscriptions.account_id	-	LAZY	false
Account	PasswordResetToken	OneToMany	password_reset_tokens.account_id	-	EAGER	false
Profile	PCMResult	OneToMany	pcm_results.profile_id	ALL	LAZY	true
Skill	Account	ManyToOne	skills.account_id	-	LAZY	false
Formation	Account	ManyToOne	formations.account_id	-	LAZY	false
Formation	Prediction	ManyToOne	formations.prediction_id	-	LAZY	false
Formation	Ticket	OneToMany	tickets.formation_id	ALL	LAZY	true
Formation	Inscription	OneToMany	inscriptions.formation_id	-	LAZY	false
Inscription	Account	ManyToOne	inscriptions.account_id	-	LAZY	false
Inscription	Formation	ManyToOne	inscriptions.formation_id	-	LAZY	false
Ticket	Formation	ManyToOne	tickets.formation_id	-	LAZY	false
PersonalityTest	Account	ManyToOne	tests_personnalite.account_id	-	LAZY	false
PCMResult	Profile	ManyToOne	pcm_results.profile_id	-	LAZY	false
Competence	CompetenceAccount	OneToMany	competence_account.competence_id	ALL	LAZY	true
CompetenceAccount	Account	ManyToOne	competence_account.account_id	-	LAZY	false
CompetenceAccount	Competence	ManyToOne	competence_account.competence_id	-	LAZY	false
Prediction	Account	ManyToOne	predictions.account_id	-	LAZY	false
Prediction	Formation	OneToMany	formations.prediction_id	ALL	LAZY	true
Recommendation	Account	ManyToOne	recommendations.account_id	-	LAZY	false
Recommendation	RecommendationItem	OneToMany	recommendation_items.recommendation_id	ALL	LAZY	true
RecommendationItem	Recommendation	ManyToOne	recommendation_items.recommendation_id	-	LAZY	false
PasswordResetToken	Account	ManyToOne	password_reset_tokens.account_id	-	EAGER	false

📖 Description Détaillée des Relations (en Français)

1. Account ↔ Profile

- **Type:** OneToOne bidirectionnelle
- **Description:** Un Account possède un et un seul Profile. La table profiles contient la colonne account_id (FK vers accounts). La contrainte UNIQUE garantit qu'un account ne peut avoir qu'un seul profil.
- **Cascade:** ALL (création, modification, suppression du profil quand l'account est supprimé)
- **orphanRemoval:** true (si le profil est retiré de l'account, il est supprimé de la base)

2. Account ↔ PersonalityTest

- **Type:** OneToMany bidirectionnelle

- **Description:** Un Account peut avoir plusieurs PersonalityTest (tests de personnalité). La table `tests_personnalite` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** ALL (tous les tests sont supprimés si l'account est supprimé)
- **orphanRemoval:** true

3. Account ↔ Skill

- **Type:** OneToMany bidirectionnelle
- **Description:** Un Account possède plusieurs Skills (compétences). La table `skills` contient la colonne `account_id` (FK vers `accounts`). Chaque `skill` appartient à un seul employé.
- **Cascade:** ALL
- **orphanRemoval:** true

4. Account ↔ Prediction

- **Type:** OneToMany bidirectionnelle
- **Description:** Un Account peut avoir plusieurs Predictions (prédictions IA). La table `predictions` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** ALL
- **orphanRemoval:** true

5. Account ↔ Formation

- **Type:** OneToMany bidirectionnelle
- **Description:** Un Account possède plusieurs Formations (formations proposées ou en cours). La table `formations` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** ALL
- **orphanRemoval:** true

6. Account ↔ Recommendation

- **Type:** OneToMany bidirectionnelle
- **Description:** Un Account peut recevoir plusieurs Recommendations. La table `recommendations` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** ALL
- **orphanRemoval:** true

7. Account ↔ CompetenceAccount

- **Type:** OneToMany bidirectionnelle (partie de la relation Many-to-Many Account ↔ Competence)
- **Description:** Un Account possède plusieurs CompetenceAccount (évaluations de compétences). La table `competence_account` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** ALL
- **orphanRemoval:** true

8. Profile ↔ PCMResult

- **Type:** OneToMany bidirectionnelle
- **Description:** Un Profile peut avoir plusieurs PCMResults (résultats de tests PCM). La table `pcm_results` contient la colonne `profile_id` (FK vers `profiles`).
- **Cascade:** ALL
- **orphanRemoval:** true

9. Formation ↔ Prediction

- **Type:** ManyToOne bidirectionnelle (Prediction OneToMany Formations)
- **Description:** Une Formation peut être liée à une Prediction (formation proposée suite à une prédiction IA). La table `formations` contient la colonne `prediction_id` (FK vers `predictions`). Une prédiction peut proposer plusieurs formations.
- **Cascade:** ALL (côté Prediction)
- **orphanRemoval:** true (côté Prediction)

10. Formation ↔ Ticket

- **Type:** OneToMany bidirectionnelle
- **Description:** Une Formation peut avoir plusieurs Tickets Jira. La table `tickets` contient la colonne `formation_id` (FK vers `formations`).
- **Cascade:** ALL
- **orphanRemoval:** true

11. Account ↔ Inscription ↔ Formation

- **Type:** Many-to-Many avec table de jointure
- **Description:** Un Account peut s'inscrire à plusieurs Formations, et une Formation peut avoir plusieurs Accounts inscrits. La table d'association `inscriptions` contient les colonnes `account_id` et `formation_id` (FKs).
- **Cascade:** Aucune (la suppression d'un account ou d'une formation ne supprime pas automatiquement les inscriptions)

12. Account ↔ CompetenceAccount ↔ Competence

- **Type:** Many-to-Many avec table de jointure et attributs supplémentaires
- **Description:** Un Account peut avoir plusieurs Competences, et une Competence peut être attribuée à plusieurs Accounts. La table d'association `competence_account` contient les colonnes `account_id` et `competence_id` (FKs), plus des champs supplémentaires (`score`, `dateEvaluation`).
- **Contrainte UNIQUE:** (`account_id`, `competence_id`) pour éviter les doublons
- **Cascade:** ALL (depuis Account et Competence)
- **orphanRemoval:** true

13. Recommendation ↔ RecommendationItem

- **Type:** OneToMany bidirectionnelle

- **Description:** Une Recommendation contient plusieurs RecommendationItems (éléments de recommandation). La table `recommendation_items` contient la colonne `recommendation_id` (FK vers `recommendations`).
- **Cascade:** ALL
- **orphanRemoval:** true

14. Account ↔ PasswordResetToken

- **Type:** OneToMany (unidirectionnelle depuis PasswordResetToken)
- **Description:** Un Account peut avoir plusieurs PasswordResetTokens (tokens de réinitialisation de mot de passe). La table `password_reset_tokens` contient la colonne `account_id` (FK vers `accounts`).
- **Cascade:** Aucune
- **Fetch:** EAGER (le token charge immédiatement l'account associé)

SECTION 5: ALL DTOs

1. AuthDto (auth module)

AuthDto.Response

Field	Type	Description
token	String	JWT access token
type	String	Token type (default: "Bearer")
id	UUID	Account ID
email	String	User email
role	String	USER or ADMIN
nom	String	Last name
prenom	String	First name
redirectUrl	String	Role-based redirect URL

AuthDto.RegisterRequest

Field	Type	Validation
lastName	String	@NotBlank
firstName	String	@NotBlank
email	String	@NotBlank, @Email
password	String	@NotBlank, @Size(min=6)
role	Account.Role	Optional (defaults to USER)

AuthDto.LoginRequest

Field	Type	Validation
email	String	@NotBlank, @Email
password	String	@NotBlank, @Size(min=6)

AuthDto.ForgotPasswordRequest

Field	Type	Validation
email	String	@NotBlank, @Email

AuthDto.ResetPasswordRequest

Field	Type	Validation
token	String	@NotBlank
newPassword	String	@NotBlank, @Size(min=6)

AuthDto.MessageResponse

Field	Type	Description
message	String	Generic message response

2. AccountDto (account module)

AccountDto.Response

Field	Type
id	UUID
username	String
email	String
firstName	String
lastName	String
department	String
position	String
hireDate	LocalDate
profilePictureUrl	String
isActive	Boolean
role	Account.Role

createdAt	Instant
updatedAt	Instant

AccountDto.CreateRequest

Field	Type	Validation
username	String	@NotBlank, @Size(min=3, max=50)
email	String	@NotBlank, @Email
password	String	@Size(min=8), @Pattern (must contain letter + digit)
firstName	String	@NotBlank, @Size(max=100)
lastName	String	@NotBlank, @Size(max=100)
department	String	@Size(max=100)
position	String	@Size(max=100)
hireDate	LocalDate	
profilePictureUrl	String	
isActive	Boolean	Default: true
role	String	

AccountDto.UpdateRequest

Field	Type	Validation
firstName	String	@NotBlank, @Size(max=100)
lastName	String	@NotBlank, @Size(max=100)
department	String	@Size(max=100)
position	String	@Size(max=100)
hireDate	LocalDate	
profilePictureUrl	String	

3. ProfileDto (account module)

ProfileDto.UpdateRequest

Field	Type	Validation
titreProfessionnel	String	@Size(max=200)
description	String	
urlPhoto	String	@Size(max=500)
experienceAns	Integer	@Min(0)
niveauEtudes	String	@Size(max=100)
lienLinkedin	String	@Size(max=500)
githubUrl	String	@Size(max=500)
cvUrl	String	@Size(max=500)

ProfileDto.Response

Field	Type	Description
id	UUID	Profile ID
accountId	UUID	Account ID
firstName	String	Read-only from Account
lastName	String	Read-only from Account
email	String	Read-only from Account
position	String	Read-only from Account
department	String	Read-only from Account
titreProfessionnel	String	Editable
description	String	Editable
urlPhoto	String	Editable
experienceAns	Integer	Editable
niveauEtudes	String	Editable
lienLinkedin	String	Editable
githubUrl	String	Editable
cvUrl	String	Editable

4. SkillDto (skills module)

SkillDto.CreateRequest

Field	Type	Validation
nom	String	@NotBlank
type	Skill.TypeSkill	@NotNull (SOFT or TECH)
niveau	Integer	@NotNull, @Min(1), @Max(5)
description	String	

SkillDto.Response

Field	Type
id	UUID
nom	String
type	Skill.TypeSkill

niveau	Integer
description	String
dateEvaluation	LocalDateTime
validee	Boolean

5. FormationDto (formation module)

FormationDto.FormationRequest

Field	Type
titre	String
description	String
type	Formation.TypeFormation
duree	Integer
fournisseur	String
url	String
dateDebut	LocalDateTime

FormationDto.FormationResponse

Field	Type
id	UUID
titre	String
description	String
type	Formation.TypeFormation
duree	Integer
fournisseur	String
url	String
statut	Formation.StatutFormation
dateProposition	LocalDateTime
dateDebut	LocalDateTime
dateFin	LocalDateTime
progression	Integer

6. PersonalityTestDto (evaluation module)

PersonalityTestDto.PersonalityTestRequest

Field	Type	Validation
typeTest	String	@NotBlank
reponses	Map<String, String>	@NotNull

PersonalityTestDto.PersonalityTestResponse

Field	Type
id	UUID
typeTest	String
reponses	Map<String, String>
resultats	String
analyseLlm	String
score	Integer
dateTest	LocalDateTime

7. PredictionDto (ai module)

PredictionDto.Response

Field	Type
id	UUID
datePrediction	LocalDateTime
analyse	String
recommandationSoft	String
recommandationTech	String
scoreConfiance	Double
statut	Prediction.StatutPrediction
formationsProposees	List<FormationDto.FormationResponse>

8. TicketResponse (jira module)

Field	Type
id	UUID
jiraKey	String
titre	String
description	String
statut	Ticket.StatutTicket
priorite	Ticket.PrioriteTicket

assignee String
urlJira String
formationId UUID
createdAt Instant
updatedAt Instant

9. DashboardDto (dashboard module)

DashboardDto.Response (Employee Dashboard)

Field	Type
accountId	UUID
nomComplet	String
firstName	String
lastName	String
nombreTests	Integer
nombreSkillsSoft	Integer
nombreSkillsTech	Integer
nombreFormationsTotal	Integer
nombreFormationsEnCours	Integer
nombreFormationsTerminees	Integer
scoreEvaluationMoyen	Double
topSkills	List<SkillDto.Response>
fomationsRecentes	List<FormationDto.FormationResponse>
demierePrediction	PredictionDto.Response

DashboardDto.EmployeeSummaryDto

Field	Type
id	UUID
firstName	String
lastName	String
position	String
department	String
formationCount	int
testCount	int
isActive	boolean
email	String

DashboardDto.AdminOverviewDto (Admin Dashboard)

Field	Type
totalEmployees	int
totalFormationsEnCours	int
totalTestsCompleted	int
totalPredictions	int
employees	List<EmployeeSummaryDto>

SECTION 6: ALL REST ENDPOINTS

1. AuthController (/api/auth) - PUBLIC

Method	URL	Auth	Request Body	Response	Description
POST	/api/auth/register	Public	AuthDto.RegisterRequest	AuthDto.Response	Register new account with role selection
POST	/api/auth/login	Public	AuthDto.LoginRequest	AuthDto.Response	Login and get JWT token
POST	/api/auth/forgot-password	Public	AuthDto.ForgotPasswordRequest	AuthDto.MessageResponse	Request password reset link
POST	/api/auth/reset-password	Public	AuthDto.ResetPasswordRequest	AuthDto.MessageResponse	Reset password with token

2. AccountController (/api/accounts)

Method	URL	Auth	Request Body	Response	Description
GET	/api/accounts	ADMIN	-	List<Account>	List all accounts
POST	/api/accounts	ADMIN	AccountDto.CreateRequest	Account	Create new account (admin only)
GET	/api/accounts/{accountId}	USER/ADMIN	-	Account	Get account by ID (ownership check)
PUT	/api/accounts/{accountId}	USER/ADMIN	AccountDto.UpdateRequest	Account	Update account details
DELETE	/api/accounts/{accountId}	ADMIN	-	Void (204)	Delete account

3. ProfileController (/api/profiles)

Method	URL	Auth	Request Body	Response	Description
GET	/api/profiles/accounts/{accountId}	USER/ADMIN	-	ProfileDto.Response	Get profile with account read-only fields
PUT	/api/profiles/accounts/{accountId}	USER/ADMIN	ProfileDto.UpdateRequest	ProfileDto.Response	Update or create profile (upsert)

4. SkillController (/api/skills)

Method	URL	Auth	Request Body	Response	Description
PUT	/api/skills/{skillId}/valider	ADMIN	-	SkillDto.Response	Validate skill (admin approval)
DELETE	/api/skills/{skillId}	USER/ADMIN	-	Void (204)	Delete skill

5. SkillAccountController (/api/skills/accounts)

Method	URL	Auth	Request Body	Response	Description
POST	/api/skills/accounts/{accountId}	USER/ADMIN	SkillDto.CreateRequest	SkillDto.Response	Create skill for account
GET	/api/skills/accounts/{accountId}	USER/ADMIN	-	List<SkillDto.Response>	Get all skills for account
GET	/api/skills/accounts/{accountId}/type/{type}	USER/ADMIN	-	List<SkillDto.Response>	Get skills by account and type (SOFT/TECH)

6. FormationController (/api/formations)

Method	URL	Auth	Request Body	Response	Description
GET	/api/formations/utilisateur/{userId}	USER/ADMIN	-	List<FormationDto.FormationResponse>	Get formations by user
GET	/api/formations/{formationId}	USER/ADMIN	-	FormationDto.FormationResponse	Get formation by ID
PUT	/api/formations/{formationId}/statut	USER/ADMIN	?statut=...	FormationDto.FormationResponse	Update formation status
PUT	/api/formations/{formationId}/progression	USER/ADMIN	?progression=...	FormationDto.FormationResponse	Update formation progress (0-100)

7. FormationAccountController (/api/formations/accounts)

Method	URL	Auth	Request Body	Response	Description
POST	/api/formations/accounts/{accountId}	USER/ADMIN	FormationDto.FormationRequest	FormationDto.FormationResponse	Create formation for account
GET	/api/formations/accounts/{accountId}	USER/ADMIN	-	List<FormationDto.FormationResponse>	Get formations by account

8. PersonalityTestController (/api/tests-personnalite)

Method	URL	Auth	Request Body	Response	Description
POST	/api/tests-personnalite/utilisateur/{userId}	USER/ADMIN	PersonalityTestDto.PersonalityTestRequest	PersonalityTestDto.PersonalityTestResponse	Submit personality test (triggers OpenAI analysis)
GET	/api/tests-personnalite/utilisateur/{userId}	USER/ADMIN	-	List<PersonalityTestDto.PersonalityTestResponse>	Get all tests by user
GET	/api/tests-personnalite/utilisateur/{userId}/dernier	USER/ADMIN	-	PersonalityTestDto.PersonalityTestResponse	Get latest test by user
GET	/api/tests-personnalite/{testId}	USER/ADMIN	-	PersonalityTestDto.PersonalityTestResponse	Get test by ID
GET	/api/tests-personnalite	ADMIN	-	List<PersonalityTestDto.PersonalityTestResponse>	Get all tests (admin only)

9. PersonalityTestAccountController (/api/tests/accounts)

Method	URL	Auth	Request Body	Response	Description
POST	/api/tests/accounts/{accountId}	USER/ADMIN	PersonalityTestDto.PersonalityTestRequest	PersonalityTestDto.PersonalityTestResponse	Create test for account
GET	/api/tests/accounts/{accountId}	USER/ADMIN	-	List<PersonalityTestDto.PersonalityTestResponse>	Get tests by account

10. PredictionController (/api/predictions)

Method	URL	Auth	Request Body	Response	Description
POST	/api/predictions/accounts/{accountId}/generer	USER/ADMIN	-	PredictionDto.Response	Generate AI prediction (calls OpenAI)
GET	/api/predictions/accounts/{accountId}	USER/ADMIN	-	List<PredictionDto.Response>	Get predictions by account
GET	/api/predictions/accounts/{accountId}/derniere	USER/ADMIN	-	PredictionDto.Response	Get latest prediction by account

11. TicketController (/api/tickets)

Method	URL	Auth	Request Body	Response	Description
POST	/api/tickets/formation/{formationId}	USER/ADMIN	-	TicketResponse	Create Jira ticket for formation
GET	/api/tickets/formation/{formationId}	USER/ADMIN	-	List<TicketResponse>	Get tickets by formation
GET	/api/tickets/{ticketId}	USER/ADMIN	-	TicketResponse	Get ticket by ID
PUT	/api/tickets/{ticketId}/statut	ADMIN	?statut=...	TicketResponse	Update ticket status (admin only)

12. DashboardController (/api/dashboard)

Method	URL	Auth	Request Body	Response	Description
GET	/api/dashboard/accounts/{accountId}	USER/ADMIN	-	DashboardDto.Response	Get employee dashboard data
GET	/api/dashboard/admin/overview	ADMIN	-	DashboardDto.AdminOverviewDto	Get admin overview dashboard (all employees)

13. TalentPredictAppication (Root Health Check)

Method	URL	Auth	Request Body	Response	Description
GET	/	Public	-	String	Health check "TalentPredict API is running!"

SECTION 7: ALL SERVICES & METHODS

1. AuthServiceImpl

- `createAccount(AuthDto.RegisterRequest) → Account` — Creates account with encoded password and role
- `getAccountById(UUID) → Account` — Retrieves account by ID, throws exception if not found
- `getAccountByEmail(String) → Account` — Retrieves account by email, throws exception if not found
- `forgotPassword(AuthDto.ForgotPasswordRequest) → String` — Generates reset token, sends email or logs token
- `resetPassword(AuthDto.ResetPasswordRequest) → String` — Validates token, updates password, marks token used

2. AccountServiceImpl

- `listAccounts() → List<Account>` — Returns all accounts in system
- `getAccountById(UUID, Account) → Account` — Gets account with policy check for viewing permissions
- `deleteAccount(UUID, Account) → void` — Deletes account with policy check for deletion permissions
- `updateAccount(UUID, AccountDto.UpdateRequest, Account) → Account` — Updates account fields with policy check

3. ProfileService

- `createProfile(Profile) → Profile` — Saves and returns new profile
- `updateProfile(UUID, Profile) → Profile` — Updates profile by ID with non-null field updates
- `updateProfileByAccountId(UUID, ProfileDto.UpdateRequest) → ProfileDto.Response` — Upsert profile by account ID
- `getProfileByAccountId(UUID) → ProfileDto.Response` — Retrieves profile by account ID, returns empty if not found
- `toResponse(Profile, Account) → ProfileDto.Response` — Converts profile entity to enriched DTO with account info
- `getProfileById(UUID) → Profile` — Gets profile by ID, throws exception if not found
- `getProfileByUser(Account) → Optional<Profile>` — Returns optional profile for account
- `getAllProfiles() → List<Profile>` — Returns all profiles
- `deleteProfile(UUID) → void` — Deletes profile by ID

4. SkillService

- `creerSkill(UUID, SkillDto.CreateRequest) → SkillDto.Response` — Creates skill for account with validee=false
- `getSkillsByAccount(UUID) → List<SkillDto.Response>` — Returns all skills for account
- `getSkillsByAccountAndType(UUID, Skill.TypeSkill) → List<SkillDto.Response>` — Returns skills filtered by type
- `validerSkill(UUID) → SkillDto.Response` — Marks skill as validated (validee=true)
- `supprimerSkill(UUID) → void` — Deletes skill by ID
- `convertToResponse(Skill) → SkillDto.Response` — Converts skill entity to DTO

5. FormationService

- `creerFormation(UUID, FormationDto.FormationRequest) → FormationDto.FormationResponse` — Creates formation with PROPOSEE status
- `getFormationsByAccount(UUID) → List<FormationDto.FormationResponse>` — Returns all formations for account
- `getFormationById(UUID) → FormationDto.FormationResponse` — Gets formation by ID, throws exception if not found
- `updateStatut(UUID, Formation.StatutFormation) → FormationDto.FormationResponse` — Updates formation status
- `updateProgression(UUID, Integer) → FormationDto.FormationResponse` — Updates progress, auto-completes at 100%
- `countFormationsByAccount(UUID) → Long` — Returns count of formations for account
- `countFormationsByAccountAndStatut(UUID, Formation.StatutFormation) → Long` — Returns count filtered by status
- `convertToResponse(Formation) → FormationDto.FormationResponse` — Converts formation entity to DTO

6. InscriptionService

- `inscrire(Account, Formation) → Inscription` — Enrolls account in formation, prevents duplicates
- `annuler(UUID) → Inscription` — Cancels inscription by setting status to TERMINEE_ANNULEE
- `terminer(UUID, Double) → Inscription` — Completes inscription with final grade and 100% progress
- `updateProgression(UUID, Integer) → Inscription` — Updates inscription progress percentage
- `getInscriptionById(UUID) → Inscription` — Gets inscription by ID, throws exception if not found
- `getInscriptionsByUser(Account) → List<Inscription>` — Returns all inscriptions for user
- `getInscriptionsByFormation(Formation) → List<Inscription>` — Returns all inscriptions for formation
- `getAllInscriptions() → List<Inscription>` — Returns all inscriptions in system

7. PersonalityTestService Uses OpenAI API

- `createTest(UUID, PersonalityTestDto.PersonalityTestRequest) → PersonalityTestDto.PersonalityTestResponse` — Creates test, generates AI analysis via OpenAI, calculates score
- `getTestsByAccount(UUID) → List<PersonalityTestDto.PersonalityTestResponse>` — Returns tests for account ordered by date descending
- `getTestById(UUID) → PersonalityTestDto.PersonalityTestResponse` — Gets test by ID, throws exception if not found
- `getLatestTestByAccount(UUID) → PersonalityTestDto.PersonalityTestResponse` — Returns most recent test or null
- `getAllTests() → List<PersonalityTestDto.PersonalityTestResponse>` — Returns all personality tests
- `calculateScore(Map<String, String>) → Integer` — Calculates score based on number of responses
- `convertToResponse(PersonalityTest) → PersonalityTestDto.PersonalityTestResponse` — Converts test entity to DTO
- **External API:** Calls `OpenAIService.analyserTestPersonnalite()` to generate AI analysis

8. PCMResultService

- `createPCMResult(PCMResult) → PCMResult` — Creates PCM result with current timestamp
- `updatePCMResult(UUID, PCMResult) → PCMResult` — Updates PCM result scores and timestamp
- `getPCMResultById(UUID) → PCMResult` — Gets PCM result by ID, throws exception if not found
- `getPCMResultsByProfil(Profile) → List<PCMResult>` — Returns all PCM results for profile
- `getAllPCMResults() → List<PCMResult>` — Returns all PCM results in system
- `deletePCMResult(UUID) → void` — Deletes PCM result by ID

9. CompetenceService

- `createCompetence(Competence) → Competence` — Creates competence with timestamps
- `updateCompetence(UUID, Competence) → Competence` — Updates competence fields with timestamps
- `getCompetenceById(UUID) → Competence` — Gets competence by ID, throws exception if not found
- `getCompetenceByNom(String) → Optional<Competence>` — Retrieves competence by name
- `getCompetenceByNomAndCategorie(String, String) → Optional<Competence>` — Retrieves by name and category
- `getAllCompetences() → List<Competence>` — Returns all competences in system
- `deleteCompetence(UUID) → void` — Deletes competence by ID

10. CompetenceUtilisateurService

- `createCompetenceUtilisateur(CompetenceAccount) → CompetenceAccount` — Creates competence-account association with timestamps
- `updateCompetenceUtilisateur(UUID, CompetenceAccount) → CompetenceAccount` — Updates score and evaluation timestamp
- `getCompetenceUtilisateurById(UUID) → CompetenceAccount` — Gets by ID, throws exception if not found
- `getCompetenceUtilisateurByUser(Account) → List<CompetenceAccount>` — Returns all competence associations for account
- `getCompetenceUtilisateurByCompetence(Competence) → List<CompetenceAccount>` — Returns all account associations for competence
- `getCompetenceUtilisateurByUserAndCompetence(Account, Competence) → Optional<CompetenceAccount>` — Returns specific association
- `getAllCompetenceUtilisateurs() → List<CompetenceAccount>` — Returns all associations in system
- `deleteCompetenceUtilisateur(UUID) → void` — Deletes association by ID

11. PredictionService Uses OpenAI API

- `genererPrediction(UUID) → PredictionDto.Response` — Gathers user data (tests, skills), generates AI prediction via OpenAI, calculates confidence score
- `getPredictionsByAccount(UUID) → List<PredictionDto.Response>` — Returns predictions ordered by date descending
- `getDernierePrediction(UUID) → PredictionDto.Response` — Returns most recent prediction or null
- `convertToResponse(Prediction) → PredictionDto.Response` — Converts prediction entity to DTO
- `convertFormationToResponse(Formation) → FormationDto.FormationResponse` — Converts formation entity to DTO
- **External API:** Calls `OpenAIService.genererPrediction()` to generate career predictions

12. RecommendationService

- `createRecommendation(Recommendation) → Recommendation` — Creates recommendation with timestamps
- `updateRecommendation(UUID, Recommendation) → Recommendation` — Updates recommendation fields with dateModification
- `getRecommendationById(UUID) → Recommendation` — Gets recommendation by ID, throws exception if not found
- `getRecommendationsByUser(Account) → List<Recommendation>` — Returns all recommendations for account
- `getAllRecommendations() → List<Recommendation>` — Returns all recommendations in system
- `deleteRecommendation(UUID) → void` — Deletes recommendation by ID

13. OpenAIService External OpenAI API Integration

- `getService() → OpenAiService` — Initializes and returns OpenAI service client (lazy initialization)
- `analyserTestPersonnalite(String, String) → String` — Generates personality test analysis including profile, strengths, improvements, soft skills
- `suggererFormations(String, String) → String` — Suggests 3-5 relevant formations based on skills and test results
- `genererPrediction(String) → String` — Generates career prediction with confidence score, skill recommendations, career opportunities
- `executePrompt(String) → String` — Executes OpenAI chat completion request with prompt and returns response
- **External API:** All analysis methods make HTTP calls to OpenAI Chat Completion API using GPT-4 model

14. JiraService External JIRA API Integration (Optional)

- `creerTicketFormation(UUID) → TicketResponse` — Creates training ticket for formation, optionally creates in JIRA if enabled
- `getTicketsByFormation(UUID) → List<TicketResponse>` — Returns all tickets for specific formation
- `getTicketById(UUID) → TicketResponse` — Gets ticket by ID, throws exception if not found
- `updateStatutTicket(UUID, Ticket.StatutTicket) → TicketResponse` — Updates ticket status, syncs with JIRA if enabled
- `creerTicketDansJira(Ticket) → void` — Creates ticket in JIRA system (placeholder implementation)
- `synchroniserStatutJira(Ticket) → void` — Syncs ticket status with JIRA (placeholder implementation)
- `determinerPriorite(Formation) → Ticket.PrioriteTicket` — Determines ticket priority based on formation type
- `convertToResponse(Ticket) → TicketResponse` — Converts ticket entity to DTO
- **External API:** Optionally calls JIRA REST API for ticket creation and status synchronization (if `jira.enabled=true`)

15. DashboardService

- `getDashboard(UUID) → DashboardDto.Response` — Returns comprehensive employee dashboard with tests, skills, formations, predictions, statistics
- `getAdminOverview() → DashboardDto.AdminOverviewDto` — Returns admin overview with aggregate data across all employees

External API Summary

Service	API	Methods	Configuration Required
OpenAI	Chat	<code>PersonalityTestService.createTest()</code>	<code>openai.api.key</code>
	Completion API (GPT-4)	<code>
PredictionService.genererPrediction()
OpenAIService (all methods)</code>	
Jira	JIRA REST API	<code>JiraService.creerTicketFormation()
JiraService.updateStatutTicket()</code>	<code>jira.enabled=true
jira.api.token
jira.url
jira.project.key</code>

SECTION 8: FRONTEND COMPONENTS

HOME MODULE

Component	Route	Guard	Display	HTTP Calls
<code>HomeComponent</code>	<code>/</code>	None (public)	Landing page, redirects authenticated users to dashboard	None (only checks local auth status)

AUTH MODULE

Component	Route	Guard	Display	HTTP Calls
LoginComponent	/auth/login	None (public)	Login form with email/password, role-based redirect	POST /api/auth/login
RegisterComponent	/auth/register	None (public)	Registration form with role selection (USER/ADMIN)	POST /api/auth/register
ForgotPasswordComponent	/auth/forgot-password	None (public)	Password recovery form, user enters email to receive reset link	POST /api/auth/forgot-password
ResetPasswordComponent	/auth/reset-password?token=...	None (public)	Password reset form with token validation for setting new password	POST /api/auth/reset-password

DASHBOARD MODULE

Component	Route	Guard	Display	HTTP Calls
UserDashboardComponent	/dashboard	authGuard	Employee dashboard: tests completed, skills count, formations progress, evaluation scores	GET /api/dashboard/accounts/{accountId}
UserProfileComponent	/profile	authGuard	User profile editor: professional title, bio, experience, education, LinkedIn, GitHub, CV URL, photo URL	GET /api/profiles/accounts/{accountId} PUT /api/profiles/accounts/{accountId}
AdminDashboardComponent	/admin/dashboard	authGuard roleGuard(['ADMIN'])	HR/Admin overview: total employees, formations in progress, tests completed, predictions, employee summaries	GET /api/dashboard/admin/overview
AdminProfileComponent	/admin/profile	authGuard roleGuard(['ADMIN'])	Admin-specific profile page with same editable fields as employee profile	GET /api/profiles/accounts/{accountId} PUT /api/profiles/accounts/{accountId}

ADMIN MODULE

Component	Route	Guard	Display	HTTP Calls
UserManagementComponent	/admin/users	authGuard roleGuard(['ADMIN'])	User management interface: view all users, update roles, delete accounts	GET /api/utilisateurs PUT /api/utilisateurs/{id} DELETE /api/utilisateurs/{id}
ReportsComponent	/admin/reports	authGuard roleGuard(['ADMIN'])	System statistics and reports: total users, formations, evaluations, tickets, active users, completion rates	GET /api/admin/stats (falls back to mock data on error)

EVALUATION MODULE

Component	Route	Guard	Display	HTTP Calls
PcmIntroComponent	/evaluation/intro	authGuard	Introduction page explaining 6 PCM personality types (Empathique, Travaillomane, Persévérant, Promoteur, Rebelle, Rêveur)	None (static content)
PcmTestComponent	/evaluation/test	authGuard	Interactive personality test with 12 questions across 6 steps, assessing PCM profile through multi-choice responses	POST /api/tests-personnalite/utilisateur/{userId}
TestResultsComponent	/evaluation/results or /evaluation/results/:id	authGuard	Displays personality test results with PCM profile, radar chart, strengths, challenges, personality description	GET /api/tests-personnalite/{testId} GET /api/tests-personnalite/utilisateur/{userId}/demier

SKILLS MODULE

Component	Route	Guard	Display	HTTP Calls
GithubAnalyzerComponent	/skills/github	authGuard	Analyzes GitHub profile to detect technical skills from repositories (currently mock implementation)	None currently (TODO: implement GitHub analysis API)
SkillComparisonComponent	/skills/comparison	authGuard	Compares user's skill levels against company averages showing positive/negative gaps for each skill	None currently (shows mock comparison data)

FORMATION MODULE

Component	Route	Guard	Display	HTTP Calls
FormationListComponent	/formations	authGuard	Lists all user's formations with filtering by status (ALL, EN_COURS, TERMINEE, PLANIFIEE)	GET /api/formations/utilisateur/{userId}

JIRA MODULE

Component	Route	Guard	Display	HTTP Calls
JiraTicketsComponent	/jira	authGuard roleGuard(['ADMIN'])	Admin-only interface to view and manage all Jira tickets with filtering by status and priority, plus sync functionality	GET /api/tickets PUT /api/tickets/{ticketId}/sync

📄 Summary by Guard Type

Public Routes (No Guards):

- Home, Login, Register, Forgot Password, Reset Password

Authenticated Routes (authGuard only):

- User Dashboard, User Profile, Evaluation (Intro, Test, Results), Skills (GitHub Analyzer, Comparison), Formations List

Admin Routes (authGuard + roleGuard[ADMIN]):

- Admin Dashboard, Admin Profile, User Management, Reports, Jira Tickets

SECTION 9: FEATURES STATUS

Feature	Status	Completion	Notes
Authentication & Authorization	✔ Done	100%	JWT authentication, role-based access (USER/ADMIN)
Password Reset Flow	✔ Done	100%	Email-based reset with token validation
User Account Management	✔ Done	100%	CRUD operations for accounts (admin only)
User Profile Management	✔ Done	100%	Editable profile with CV, LinkedIn, GitHub links
Personality Tests (PCM, MBTI, DISC)	✔ Done	100%	Submit tests, OpenAI analysis, results display
Skills Management (Soft/Tech)	✔ Done	100%	Create, validate, delete skills with 5-level system
AI Predictions	✔ Done	100%	OpenAI-powered career predictions and recommendations
Formations Management	✔ Done	100%	Create, track, update status and progression
Jira Ticket Integration	◇ Partial	60%	Database implementation done, Jira API integration is placeholder (requires <code>jira.enabled=true</code>)
Dashboard (Employee)	✔ Done	100%	Statistics, top skills, recent formations, latest prediction
Dashboard (Admin/RH)	✔ Done	100%	Overview of all employees, formations, tests, predictions
PCM Results Storage	✔ Done	100%	Store PCM test results linked to profiles
Competence Evaluation	✔ Done	100%	Many-to-many association between accounts and competences with scores
Inscriptions (Enrollments)	✔ Done	100%	Many-to-many enrollment system for formations
Recommendations	✔ Done	100%	AI-generated recommendations with items
GitHub Skills Analyzer	✗ Missing	0%	Frontend page exists but API integration not implemented
Skills Comparison	✗ Missing	0%	Frontend page exists with mock data, backend logic not implemented
Admin Reports	◇ Partial	50%	Frontend exists, <code>/api/admin/stats</code> endpoint missing (falls back to mock)
User Management (Admin)	◇ Partial	70%	Frontend exists, uses <code>/api/utilisateurs</code> (non-standard endpoint, should be <code>/api/accounts</code>)
Formation Tickets Sync	◇ Partial	40%	Ticket entities and endpoints exist, actual Jira API calls are placeholders
Camunda BPM Workflows	✗ Missing	0%	Dependencies disabled in pom.xml, no BPMN processes implemented
Email Service (Password Reset)	✔ Done	100%	Spring Mail configured, sends reset emails (or logs if mail not configured)
JWT Token Management	✔ Done	100%	Token generation, validation, role-based authorization
CORS Configuration	✔ Done	100%	Configured for frontend communication
Global Exception Handling	✔ Done	100%	Centralized exception handling with consistent error responses
Validation & Constraints	✔ Done	100%	Jakarta Validation on all DTOs, database constraints on entities

⚠ Implementation Gaps & Recommendations

- Admin Statistics Endpoint Missing
 - Frontend `/admin/reports` expects GET `/api/admin/stats`
 - **Recommendation:** Implement endpoint or redirect to existing dashboard endpoints
- User Management Endpoint Mismatch
 - Frontend uses `/api/utilisateurs` but standard endpoint is `/api/accounts`
 - **Recommendation:** Create `/api/utilisateurs` alias or update frontend to use `/api/accounts`
- GitHub Skills Analyzer
 - Frontend component exists but no backend API
 - **Recommendation:** Implement GitHub API integration to analyze repositories and extract skills
- Skills Comparison
 - Frontend shows mock data, no backend logic for company-wide skill averages
 - **Recommendation:** Implement aggregation logic to calculate average skill levels across employees
- Jira API Integration
 - Placeholder implementation, requires actual Jira REST API calls
 - **Recommendation:** Complete `JiraService.creerTicketDansJira()` and `synchroniserStatutJira()` methods
- Camunda BPM Workflows
 - Dependencies commented out, no workflows implemented
 - **Recommendation:** Either implement approval workflows or remove references from documentation

📊 OVERALL_COMPLETION: 82%

Breakdown:

- Core Features (Auth, Profiles, Skills, Tests, Formations, Predictions): 95%
- Admin Features (Dashboard, User Management): 75%
- External Integrations (OpenAI, Jira, GitHub): 70%
- Advanced Features (Workflows, Analytics): 50%

END OF REPORT

Generated: 2026-03-05

Project Path: C:\Users\rahma\Desktop\TalentPredict

Report Format: Markdown (can be converted to PDF using tools like Pandoc, Markdown PDF extension, or online converters)